TECHNICAL

Data Structures Notes & Algorithm Patterns

Linear Data Structures & Sorting Complexity

1. Complexity Analysis & Big-O Notation

- Array Access: O(1) time; Insertion/Deletion: O(n) worst-case due to element shifting.
- Linked List: Singly vs. Doubly vs. Circular; Insertion at head O(1), Search O(n).
- Stack (LIFO): push, pop, peek O(1); Applications: Expression parsing, backtracking.
- Queue (FIFO): enqueue, dequeue O(1); Variations: Circular Queue, PriorityQueue (Heap-backed).

2. Sorting & Searching Comparison Table

- QuickSort: Best O(n log n), Worst O(n^2), Space O(log n) recursive stack. In-place, unstable.
- MergeSort: Guaranteed O(n log n) in all cases, Space O(n). Stable, suitable for linked lists.
- HeapSort: Guaranteed O(n log n), Space O(1). In-place, unstable.
- Binary Search: O(log n) time on sorted collection; Two-pointer and sliding window patterns.

TECHNICAL

Data Structures Notes & Algorithm Patterns

Non-Linear Structures & Graph Traversals

3. Trees & Binary Search Trees (BST)

- Traversals: In-order (Left, Root, Right -> Sorted for BST), Pre-order (Root, L, R), Post-order (L, R, Root).
- Balanced Trees: AVL and Red-Black Trees guarantee O(log n) search/insertion/deletion.
- Trie (Prefix Tree): Optimal for autocomplete, prefix search, and dictionary word lookup.

4. Graphs & Dynamic Programming

- Graph Representations: Adjacency Matrix O(V^2) space vs. Adjacency List O(V + E) space.
- Traversals: BFS uses Queue (Shortest path in unweighted graphs); DFS uses Stack/Recursion.
- Shortest Path: Dijkstra's Algorithm O((V + E) log V) with Min-Heap; Bellman-Ford for negative weights.
- Dynamic Programming: Optimal Substructure + Overlapping Subproblems (Memoization vs. Tabulation).